FIFO, LRO, Optimal

```
#include <stdio.h>

#define MAX 50

void printFrames(int f[], int n)
{
    printf("[");
    for (int i = 0; i < n; i++)
    {
        if (f[i] != -1) printf("%d", f[i]);
        else printf(" ");
        if (i != n - 1) printf(" ");
    }
    printf("]\n");
}

int search(int f[], int n, int p)
{
    for (int i = 0; i < n; i++)
        if (f[i] == p) return 1;
    return 0;
}

// FIFO
void FIFO(int p[], int n, int nf)
{
    int f[MAX], faults = 0, next = 0;

    for (int i = 0; i < nf; i++) f[i] = -1;
```

```
    printf("\n--- FIFO Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        if (!search(f, nf, p[i]))
        {
            f[next] = p[i];
            next = (next + 1) % nf;
            faults++;
        }

        printf("Page %d -> ", p[i]);
        printFrames(f, nf);
    }

    printf("Total Page Faults (FIFO): %d\n", faults);
}

// LRU
void LRU(int p[], int n, int nf)
{
    int f[MAX], time[MAX], faults = 0;

    for (int i = 0; i < nf; i++)
        f[i] = time[i] = -1;

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0, pos = 0;
```

```
for (int j = 0; j < nf; j++)
{
    if (f[j] == p[i])
    {
        found = 1;
        time[j] = i;
    }
}

if (!found)
{
    for (int j = 0; j < nf; j++)
    {
        if (f[j] == -1)
        {
            pos = j;
            break;
        }

        if (time[j] < time[pos])
            pos = j;
    }

    f[pos] = p[i];
    time[pos] = i;
    faults++;
}

printf("Page %d -> ", p[i]);
printFrames(f, nf);
```

```
    }

    printf("Total Page Faults (LRU): %d\n", faults);
}

// Optimal
void Optimal(int p[], int n, int nf)
{
    int f[MAX], faults = 0;

    for (int i = 0; i < nf; i++) f[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        if (!search(f, nf, p[i]))
        {
            int pos = -1, farthest = i;

            for (int j = 0; j < nf; j++)
            {
                int k;

                if (f[j] == -1)
                {
                    pos = j;
                    break;
                }

                for (k = i + 1; k < n; k++)
```

```
                if (f[j] == p[k]) break;

            if (k > farthest)
            {
                farthest = k;
                pos = j;
            }
        }

        f[pos] = p[i];
        faults++;
    }

    printf("Page %d -> ", p[i]);
    printFrames(f, nf);
  }

  printf("Total Page Faults (Optimal): %d\n", faults);
}

int main()
{
  int n, nf, p[MAX];

  printf("Enter number of pages: ");
  scanf("%d", &n);

  printf("Enter page reference string:\n");
  for (int i = 0; i < n; i++)
    scanf("%d", &p[i]);
```

```
    printf("Enter number of frames: ");
    scanf("%d", &nf);

    FIFO(p, n, nf);
    LRU(p, n, nf);
    Optimal(p, n, nf);

    return 0;
}
```

Output:

```
Om-1WA24CS198Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7    ]
Page 0 -> [7 0  ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7

Process returned 0 (0x0)   execution time : 43.650 s
```